

The aim of this project is to build a **Sentiment Classification system** that can automatically determine whether a given piece of text expresses **positive or negative emotion**. Instead of using modern datasets like IMDb, the goal was to analyze **classical English literature** — specifically *Shakespeare's Hamlet* — using **NLTK (Natural Language Toolkit)** and a **Long Short-Term Memory (LSTM)** neural network.

Dataset and Tools Used

1. Dataset: Project Gutenberg – Shakespeare's Hamlet

- The text data was obtained from the **Gutenberg corpus** available in **NLTK**.
- *Hamlet* contains around **2,355 sentences**, out of which **923 sentences** were automatically labeled for sentiment.

2. Libraries and Tools

Tool	Purpose
NLTK (Natural Language Toolkit)	Used for text preprocessing, tokenization, stopwords removal, and accessing the Gutenberg corpus.
Project Gutenberg Corpus	Provided the classical English text (<i>Shakespeare's Hamlet</i>).
VADER (Valence Aware Dictionary and sEntiment Reasoner)	Used to automatically label each sentence with a sentiment score (positive or negative).
TensorFlow / Keras (LSTM Model)	Used to build and train the deep learning model for sentiment classification.
Python	Programming language used for all implementation and analysis.

Methodology

Step 1 Data Loading

- Loaded *Shakespeare's Hamlet* using `nltk.corpus.gutenberg.raw('shakespeare-hamlet.txt')`.
- Text was split into individual **sentences** using `sent_tokenize()`.

Step 2 Sentiment Labeling using VADER

- VADER was used to assign a **sentiment polarity score** (compound value) to each sentence.
- Sentences with scores:
 - $\geq +0.05 \rightarrow$ Positive (label = 1)
 - $\leq -0.05 \rightarrow$ Negative (label = 0)
 - Neutral sentences were discarded.
- Final labeled dataset: **923 sentences**.

Step 3 Text Preprocessing

- Converted text to lowercase.
- Removed punctuation, special symbols, and stopwords.
- Tokenized sentences into words using `word_tokenize()`.

Step 4 Data Preparation for LSTM

- Used **Keras Tokenizer** to convert words into integer sequences.
- Applied **padding** using `pad_sequences()` to make all sequences equal length (30 tokens each).
- Split data into **training (80%)** and **testing (20%)** sets.

Step 5 Model Building

An **LSTM (Long Short-Term Memory)** model was built using Keras Sequential API with the following layers:

1. **Embedding Layer (128 dimensions)** – Converts word indices to dense word vectors.
2. **LSTM Layer (128 units)** – Learns sequential dependencies and contextual sentiment.
3. **Dense Output Layer (Sigmoid activation)** – Outputs probability for binary sentiment classification.

Optimizer: Adam (learning rate 0.001)

Loss Function: Binary Cross-Entropy

Epochs: 5

Batch Size: 32

Step 6 Model Training

- The model was trained for **5 epochs** with **validation split = 20%**.
- Training accuracy improved from **61% → 87.8%**, while validation accuracy reached **69.6%**, indicating strong learning and moderate generalization.

Step 7 Model Evaluation

- On unseen test data, the model achieved **Test Accuracy = 72.97%**.
- It performed well despite the complexity of Shakespearean language.

Results

Metric	Value
Training Accuracy	87.8%
Validation Accuracy	69.6%
Test Accuracy	72.97%

Example Predictions:

Sentence	Predicted Sentiment
<i>"I love thee more than words can wield the matter."</i>	Negative (0.05) (<i>archaic wording</i>)
<i>"O wretched state! O bosom black as death!"</i>	Negative (0.06)

Observation: The model correctly captured emotional tone, though older English phrasing sometimes affected the labeling accuracy.

Understanding Key Components

NLTK (Natural Language Toolkit)

Used for text preprocessing — tokenization, stopwords removal, and corpus handling.

Project Gutenberg

A free digital library of classic texts (public domain). Provided *Hamlet* for analysis.

VADER (Valence Aware Dictionary and sEntiment Reasoner)

A lexicon-based sentiment analyzer that already contains **pre-scored words**.

It assigns a **compound score** to each sentence based on known positive/negative word values.

If a word is **not in its lexicon**, VADER treats it as **neutral (score = 0)**.

LSTM (Long Short-Term Memory)

A type of **Recurrent Neural Network** (RNN) that learns the meaning of words in sequence.

It can remember long-term dependencies, making it ideal for **sentiment classification** where **word order and context** matter.

Why I Chose This Method

Aspect	Reason
LSTM over traditional ML models	Traditional models (like Logistic Regression or Random Forest) treat text as bag-of-words, ignoring order and context. LSTM understands the sequence of words , capturing meaning like “not good” ≠ “good.”
VADER + LSTM hybrid	VADER helps generate sentiment labels quickly without manual labeling, and LSTM learns deeper emotional patterns beyond simple word scores.
Gutenberg (Hamlet)	Using a classical text like <i>Hamlet</i> shows the model’s robustness on complex, poetic English rather than standard modern data.
NLTK Integration	Simplifies text cleaning, tokenization, and corpus access — providing an end-to-end NLP pipeline.

Observations

- The dataset from *Hamlet* is small and contains archaic language, making classification more challenging.
- The LSTM model still learned strong patterns of emotional polarity.
- Words not present in VADER’s dictionary were treated as neutral, explaining slight accuracy variation.
- Combining VADER and LSTM created a semi-supervised workflow — VADER labeled, LSTM learned.

Conclusion

This project successfully demonstrated **Sentiment Classification using LSTM** on a **classical text corpus** (Shakespeare's *Hamlet*) using **NLTK and VADER**.

The system achieved **~73% test accuracy**, showing that LSTM effectively captures sentiment even from complex, old-style English.

By combining **rule-based sentiment labeling (VADER)** with **deep learning (LSTM)**, this project illustrates how modern AI can understand emotions in historical literature.

Future Scope

- Use larger classical datasets (multiple Shakespeare plays or modern literature).
- Incorporate **pre-trained embeddings (GloVe or Word2Vec)** for better semantic understanding.
- Explore **Bidirectional LSTM** or **Transformer-based models (BERT)** for improved accuracy.
- Create a **visual sentiment timeline** across the play (to see emotional highs/lows in acts).

Final Takeaway

"This project bridges classical literature and modern AI — using NLTK and LSTM to teach a machine how to feel Shakespeare's emotions."